

Week 2 - Modeling foundations

In this assignment, you will take the next step in the machine learning workflow. The goal of this assignment is to introduce you to the core ideas of supervised learning. You will work with the Titanic passenger dataset, an open dataset that is commonly used for educational purposes in machine learning.

The assignment is divided into two parts. In the first part, you will explore the dataset, prepare it for modeling, and train a classification model using a machine learning library. In the second part, you will go one level deeper and implement a machine learning model from scratch. Instead of relying on libraries with pre-build abstractions, you will reconstruct the components of the algorithm in Python code. This will help you understand how predictions are generated, how model parameters are learned, and what role the optimization plays in the training process.

Part 1: Classification model using scikit-learn

Dataset: In this assignment, you will work with the Titanic passenger dataset. This dataset contains information about the individual passengers, including age, sex, ticket class, and whether they survived the disaster. The task is to build a model that can predict whether a passenger survived based on these features.

The dataset can be found in the data folder as `titanic.csv`.

Exploration: Before you start with modeling, it is important to become familiar with the structure and content of the dataset. Inspect the dataset and create summaries, identify data types, check for any data quality issues, and create visualizations that help you to explore the dataset.

Preparation: The next step is to create a clean and consistent dataset that can be used as input for model training. This involves selecting relevant features and transforming the features into a suitable numerical representation.

Model training: Using the prepared dataset, train a classification model (logistic regression) using the scikit-learn library.

Evaluation: Finally, evaluate the performance of your model. Explore multiple evaluation metrics and reflect on what each of them reveals about the model.

Part 2: Logistic regression from scratch

In the first part of the assignment, you trained a model using a high-level library. In this part, you will go one step further and implement logistic regression yourself using only Numpy.

The goal of this exercise is not to build a production-ready model, but to develop an intuition for how machine learning models work internally. In applied machine learning, you will almost always rely on optimized libraries such as scikit-learn.

Training a model means adjusting its parameters so that its predictions better match the observed data. In logistic regression, the parameters are the weights and the bias. The weights determine how strongly each input feature influences the prediction, while the bias acts as an offset that shifts the prediction up or down independently of the features.

A logistic regression model is built from a sequence of components: a linear model to compute a score, a sigmoid function to convert that score into a probability, a loss function to measure error, and gradients to determine how the parameters should be updated. These components are combined in a training loop that iteratively adjusts the weights and bias to improve the model's predictions.

Follow the implementation steps in the notebook.

Tips for implementation

- Implement the functions in order and test each step before moving on.
- Always use the probabilities that the model predicts, not the final classification values.
- Use small datasets first to verify correctness.
- Play close attention to the array shapes when using Numpy operations.
- Use `np.clip` in the loss function to avoid numerical issues with logarithms.
- Print the loss values to debug the training. If the loss does not decrease, check the gradient calculation and learning rate.
- Keep your implementation simple!

Acceptance criteria

- The implementation is provided as a Jupyter notebook.
- The notebook executes from start to finish without errors.
- The code cells are logically structured and contain a short description using markdown cells.
- **Part 1:**
 - Summaries and visualizations of the dataset are created.
 - The dataset is prepared for modeling.
 - A logistic regression model is trained using scikit-learn.
 - Model performance is evaluated using multiple metrics.
- **Part 2:**
 - The model components are implemented as separate functions.
 - The training loop tracks the loss over time.
 - The loss decreases during training.

- The logistic regression implementation is applied to the cleaned dataset from Part 1.